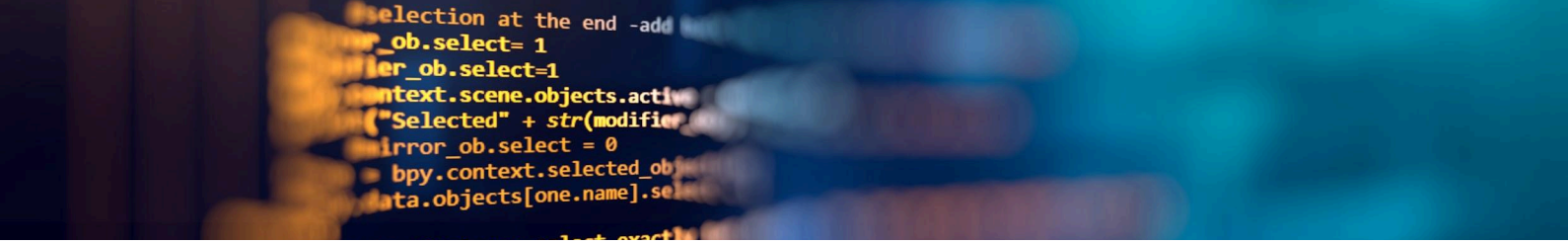


THE UNIVERSITY of EDINBURGH

edinburgh genomics

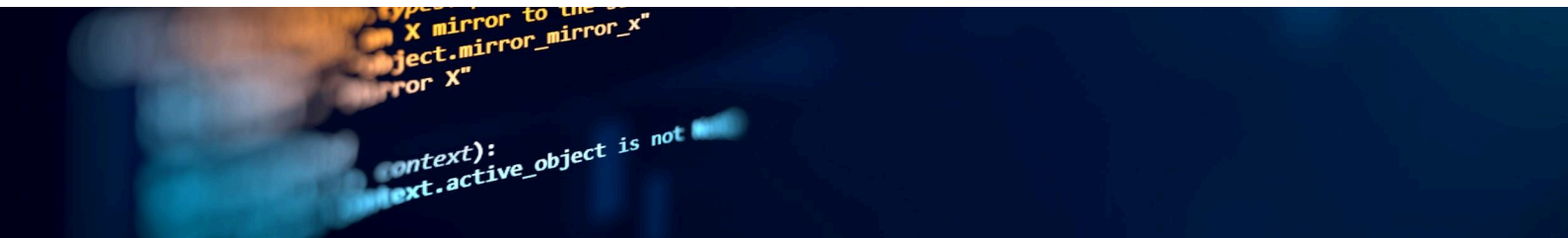
s

$ Linux for GENOMICS

Instructors:

Tim Booth - Bioinformatician and Programmer (*tim.booth@ed.ac.uk*)

Heleen de Weerd - Bioinformatician and Analyst



Course Book Contents

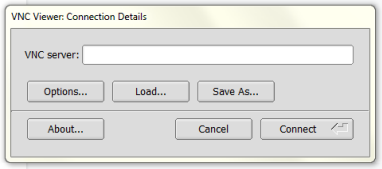
Connecting to our training virtual machines (VMs)

1. First, you will need a VNC (virtual network computing) viewer running on your PC or Mac. 'TigerVNC' is our preferred choice. Please follow this link to download it now:

https://sourceforge.net/projects/tigervnc/files/stable/1.13.1/

For Windows you want the file called ***vncviewer64-1.13.1.exe****, because you only need the viewer not the full server.*

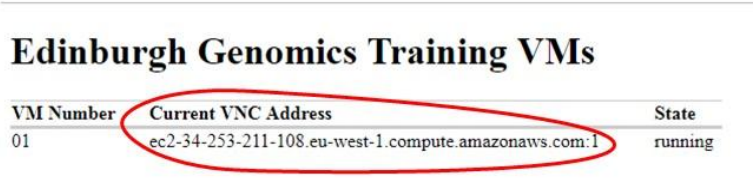
2. Once TigerVNC is successfully downloaded, click on it to run. You will be greeted with a grey dialogue box asking for a VNC server connection address.



3. To discover the connection address your personal VM for the course, go this page:

tinyurl.com/egtraining

… and copy the 'Current VNC Address' next to your VM number from this page into your TigerVNC box.



Edinburgh Genomics Training VMs

VM Number	Current VNC Address	State
01	ec2-34-253-211-108.eu-west-1.compute.amazonaws.com:1	running

Clicked 'Connect' and when prompted enter the password `EdG3n0m1cs`. You should be greeted by a Linux desktop.

Welcome to your VM for the course! This VM is yours to play with throughout the course, it is a 'sandbox' that can be reset or restarted in an instant. **You can't break anything or delete anything important.**

Your VM will be shut down and rebooted overnight, so remember to **save your work and close down applications at the end of each day**. Any files or notes you want to have access to at home can be sent to yourself by email or dropped into an online folder, Google Drive, OneDrive etc.

Steps 2-3 will have to be repeated every morning of a multi-day course.

Introduction

Genomic studies produce vast amounts of data, usually in the form of very large text* files. Commands available on the Linux command line are particularly suited for working with such files and it is therefore arguably one of the most important tools in a bioinformatician's toolkit. The command-line enables one to search, manipulate and distil large text files that are difficult or impossible to handle with applications like Word or Excel. By chaining commands you can write pipelines to perform certain tasks, and run bioinformatics software for which no web or GUI interface is available.

** We'll explore the distinction between "text files" and "binary files" later.*

In this course you will learn to:

- Navigate the file system using the Linux command shell (chapters 1-6)
- Use basic shell commands to view and filter data (chapters 7-11)
- Run some key bioinformatics tools and combine them into scripts (chapters 12-16)

You will use the following tools:

- Ubuntu Linux Distro with XFCE4: https://www.xubuntu.org
- Bash: https://en.wikipedia.org/wiki/Bash_(Unix_shell)
- GNU coreutils: https://www.gnu.org/software/coreutils

How to use this tutorial:

Example commands are shown like this:

```
$ ls
```

Type the command (without the `$`, which represents the prompt) into the terminal and press the `[Enter]` key to run it. For some example commands the expected output is shown directly underneath the command. When output ends with `...`, it means that only a part of the real output is shown.

Unless stated otherwise, the examples all assume that you start work in the directory `/home/training/Linux`.

Extra information / tips

Information that is useful but not essential to the tutorial is shown like this.

In various places in the tutorial there are exercises with sample answers.

1. The shell and commands

The command-line interface in Linux is provided by a program called the **shell**. The most common shell program found in Linux distributions is **Bash** (the name is a pun on the original developer Stephen Bourne - **B**ourne **A**gain **Sh**ell). Bash is a command processor, i.e. a program that takes commands and passes them on to the operating system kernel, which executes them.

Bash typically runs in a text window, where the user types **commands** that cause actions and views outputs. When using a graphical interface like XFCE Desktop, we run a program called a terminal emulator (also called terminal window or terminal) to interact with the shell. Different Linux distributions have different terminal emulators, but they all do the same thing, they give access to the shell. Bash can also read commands from a text file, called a **shell script**, in which case it doesn't necessarily need a terminal.

➔ Open a terminal window now, by clicking the Terminal Emulator icon on the left on your Linux desktop

You can move and resize the terminal window using the mouse, or maximise it using the maximise icon, and increase and decrease the text size by selecting 'View > Zoom In' and 'View > Zoom Out' from the menu bar. You can open as many terminals as you like.

What you will see in your terminal window is the **command prompt**, which consists of a user name (in this case, `training`), the name of the machine this user is working on (eg. `vm-01`), and the name of the current directory (~), followed by a **`$`** sign that marks the end of the prompt:

```
training@vm-01:~$
```

For the sake of simplicity, and because the prompt can change, in our examples we will just use the `$` sign to represent the entire command prompt. The basic structure of a Linux command is:

```
command [option(s)] [argument(s)]
```

- `command`: the operation or program you want Linux to execute
- `options` (also called `flags` or `switches`): modify the way the command works
- `arguments`: filenames or other targets that direct the action of the command

Note that options and arguments are not always needed, when a command has some default behaviour.

Commands are executed by typing in the command at the command prompt, followed by pressing the `[Enter]` key.

Here are two examples:

The **`date`** command simply displays the current date and time in the terminal. If run with no options it shows the local time, but if run with the "`-u`" **option** it shows UTC. By convention, options to Linux commands are always preceded by hyphens like this, while arguments are not.

```
$ date
Fri 19 Jun 15:35:03 BST 2020
$ date -u
Fri 19 Jun 14:35:20 UTC 2020
```

Note: You must put a space between the command name ("date") and the option ("-u"), but you must not put a space between the hyphen and the letter "u". Also, you must use all lower case.

➔ In your terminal, run the two "date" commands as shown.
➔ Break the rules in the note above - what happens?

Course Materials

Before we can go any further we need some demo data to play with. This can be loaded onto you machines by unpacking a compressed file from a shared location to you own VM.

We do this using the tar command. We'll explore how this works in a little more detail later in the course so for now just trust us and type the following into the terminal and hit enter.

```
$ tar -xvf /mnt/s3fs/tar/linux_course.tar.xz
```

OK, you should now have a folder called Linux. So, how do we view the directories we have?

The `ls` (list) command lists the contents of the current directory:

```
$ ls
Desktop    Linux     Public  R_for_Genomics  Videos
Documents  Music     R       Templates       bin
Downloads  Pictures  RNAseq  Variants        igv
```

If you give a directory name as an **argument** to the **`ls`** command it will list that directory instead.

```
$ ls Linux
Homo_sapiens.GRCh38.dna.primary_assembly.fa
linux_intro_exercises_final2.pdf  toy.sam
Mus_musculus_snps_19_1_20000000.vcf          mouse_exons.bed
```

Previous commands you have run can be recalled using the `[Up arrow]` key. This is particularly useful to run a similar command without writing the whole command out again.

You can move the cursor through a command using the `[Right arrow]` and `[Left arrow]` keys. To jump directly to the beginning or end of a command, use the `[Ctrl]+[A]`

and [Ctrl]+[E] keys, respectively. Note that after editing the command you **don't** have to move the cursor to the end before you press [Enter].

➔ Run the ls commands above
➔ Compare the result with what you see in the graphical file explorer
➔ Now add the "-l" option. What change does this make?

Previous commands can also be listed using the command **history** :

```
$ history
1 date
2 date -u
5 date - U
3 ls
4 ls -l Linux
```

You can clear the terminal window using the **clear** command, and terminate the current Bash session using the **exit** command (but let's not do that now).

The mouse

In the terminal, the mouse does not move the input cursor - you must use the arrow keys for that.

Having said this, the mouse can still be handy. Besides using the mouse to resize and scroll the terminal window, you can use it to **copy text**.

➔ To this end, drag your mouse pointer over some text while holding down the left button, which should highlight the text. Subsequently, right-click and select 'copy'.
➔ Then open a second terminal, right click in the second terminal and select 'paste'. The highlighted text should now have been copied to the command line.
➔ Use the **exit** command to close this second terminal

In this section we have learned the following commands:

date	show the date and time
ls	list contents of a directory
history	show previously used commands
clear	clear terminal window
exit	terminate current session

2. Getting help within the shell

There are several ways to find out what a command does and what options are available for that command.

Most commands can be run with the **`--help`** or **`-h`** option to get a brief description of the command, e.g.:

```
$ ls --help
```

Some commands don't have a `--help` or `-h` option. For these commands, you can try the **`help`** function instead:

```
$ help cd

$ help help
```

If you run the second of those you will see that **`help`** "Display[s] information about builtin commands." So it only tells you what Bash knows about the command, and Bash only knows about basic commands.

```
$ man ls
```

The **`man`** command opens up a **manual page** (or "manpage") for a particular command. Manpages generally contain more detailed information than you'll get with the `--help` or `-h` flags, plus you can get information on just about any command on the system.

You can scroll through the man page one line at a time using the `[Down arrow]` and `[Up arrow]` keys, or one screen at a time using the `[Page Down]` key or space bar, and the `[Page Up]` or `[B]` key.

You can search in the manpage by typing a `/` followed by your search term and pressing `[Enter]`. This will highlight all occurrences of your search term in the manpage. You can use `[N]` and `[Shift]+[N]` to jump forwards and backwards between the highlighted occurrences.

Some manpages are more informative than others, but one tip when encountering a new command is to skip down to the bottom, as there are often examples of how to use the command near the end.

To quit the manpage, use the `[Q]` key. (`[Ctrl]+[C]` doesn't work in this case)

Stopping a command with [Ctrl]+[C]

When you want to kill (ie. quit immediately from) a job you're running in the terminal, use the `[Ctrl]+[C]` key combination.

Often this key combination fixes the problem when your terminal stops responding or when for some reason you've lost the command prompt. It can also be used to quickly abandon the current command you are typing and bring up a fresh prompt.

Note that in many applications like word processors this key combination is used to copy text, but in the shell it is used to stop programs.

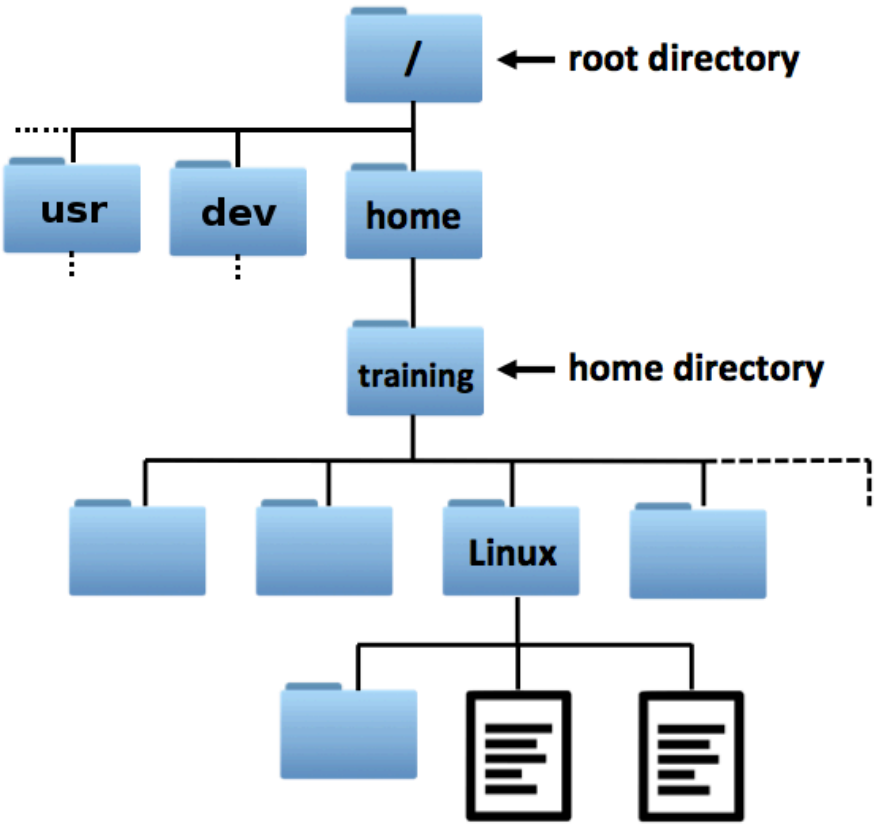
In this section we have learned the following commands:

command `--help`	show help for *command*
`help` *command*	show help for *command*
`man` *command*	show manual page for *command*
`[Ctrl]+[C]`	kill the current job

3. Files and directories

Files on the Linux system are grouped into **directories** (also known as folders, just as in a Windows or Mac environment). The directories are arranged in a hierarchical tree structure. At the top is the **root directory** `"/"` that holds everything else. The root directory can contain **files** and **subdirectories**. The subdirectories in turn can contain files and subdirectories, and so on, and so on. It's vital to understand that you will always be in a particular working directory when using the shell, and any commands like `ls` will run in that context.

Below is a small part of the directory structure on our system.



To identify any file or directory we can give a **path** to that file by starting from the root and working down. These paths are written as a list of directory names separated by forward-slash characters - eg. `/home/training/Linux/mouse_exons.bed`.

When you first start a shell on a Linux system, your **working** (or **current**) **directory** will be your **home directory**. Most typically, your home directory will be called `/home/username`, for example `/home/johndoe` or in our case `/home/training`. If you refer to the name of a file and don't provide a path then Linux will look for the file in the working directory.

The `pwd` (print working directory) command shows the current directory where Bash is working:

```
$ pwd
/home/training
```

That is, the `training` directory within the `home` directory within the root directory. Note that, although there is a directory named simply `home`, this is **not** your home directory, but `/home/training` is!

Best practices for choosing good file and directory names

1. Everything in Linux is **case sensitive**. If in doubt, stick to **lower case**.

The filename `myfile` is **not** the same as `Myfile`, `MyFile`, or `myFile`. Similarly, typing the command `pwd` is different to typing `PWD`.

2. File and directory names should **only contain letters**, **numbers**, **hyphens**, **underscores**, and **full stops**.

3. File and directory names should **not contain any spaces**.

In contrast to graphical environments, the command-line interface can become cumbersome when your file names contain spaces or special characters. For example, the shell will consider `my file` to be two files, i.e. "my" and "file". To use this filename, you will need to enclose the entire filename in quotation marks or escape the space with a backslash, so that the shell understands that the space is part of the name:

`“my file” or my\ file`

So, it's best to try to avoid spaces and unusual characters altogether, even though Linux will not prevent you from putting them into file names.

In this section we have learned the following command:

pwd	print working directory

4. Navigating the file system

With the **cd** (change directory) command you can go to another directory:

```
$ cd Linux
$ pwd
/home/training/Linux
```

Note that the actual command prompt now has changed, from

training@vm-01:~$ to training@vm-01:Linux$.

Tab completion

You can save yourself a lot of typing by using **tab completion**. Tab completion means that partially typed file names are automatically filled in by pressing the [Tab] key. In the example above, if you type only cd L and then press the [Tab] key, your command will automatically be completed to cd Linux/ because Linux is the only subdirectory starting with the letter L. If nothing happens after pressing the [Tab] key, it may mean that there are several options to complete your command. For example, if you type cd D and then press the [Tab] key, nothing will happen, because there are several subdirectories that start with the letter D. Pressing the [Tab] key again will show all these options: Desktop/ Documents/ Downloads/. For successful tab completion, in this case, you have to type some more letters.

➔ Practise some tab completions, and try to use it from now on, as it will save you a lot of time and also will prevent you from making typos!

Use **cd ..** to go to the directory above your current directory (i.e. its **parent directory**). In this case, this puts us back in the home directory:

```
$ cd ..
$ pwd
/home/training
```

And **../..** to go two directories above your current directory. Now this takes us right up to the root:

```
$ cd ../..
$ pwd
/
```

To go straight back to your home directory, use `cd` without any argument:

```
$ cd
$ pwd
/home/training
```

You may have noticed that the prompt uses a `~` character to indicate you are in your home directory. This is a standard shell abbreviation, so **`cd ~`** is an equivalent command that takes you directly home.

An important concept is that of **absolute and relative paths**. An **absolute path** specifies a location from the root of the filesystem, and starts with a **`/`**, while a **relative path** specifies a location starting from the current location and does not start with a **`/`**.

To demonstrate, first go the `Linux` directory:

```
$ cd Linux
```

If you now want to go from the `Linux` directory to the `Downloads` directory in a single step, you can do this by specifying the **absolute path**, starting from the root directory (have a look again at the directory structure in part 3, if needed):

```
$ cd /home/training/Downloads
```

Or by specifying the **relative path**, starting from the directory where you currently are (i.e. `/home/training/Linux`):

```
$ cd ../Downloads
```

Here, because we are already in the `Linux` directory we have to construct a path that goes back up to the parent directory in order to find the `Downloads` directory. But if we started from somewhere else we'd need to construct a different route. So, the **absolute path** to a particular directory or file **will always be the same**, no matter where you are in the directory structure at that moment, while the **relative path varies** depending on where you are in the directory structure.

One common gotcha is this:

```
$ ls Linux
...list of files, as expected...
$ cd Linux
$ ls Linux
ls: cannot access 'Linux': No such file or directory
$ cd Linux
bash: cd: Linux: No such file or directory
```

If you are already working inside a directory, then referring to that directory by name doesn't work, as shown above. This is because the rules of relative paths say that they are relative to the working directory, and here the Linux directory is not **within** the working directory it **is** the working directory - all directories are found within their parent directory (`..`), by definition. Of course, it is possible to have another directory named Linux within the directory named Linux, but in this case there is no such directory.

As we already have seen, the tilde symbol `~` is a shorthand for the home directory (`/home/training`), so:

```
$ cd ~/Linux
```

will work no matter what your working directory is, and is the same as:

```
$ cd /home/training/Linux
```

Finally, the single dot `.` is a shorthand for the working (current) directory, just as `..` is the parent directory. This seems pretty redundant now but we'll find a use for it later.

Once again, absolute paths always start with a `/` (because they start from the root directory), while relative paths start from the working directory. Paths starting with `~/` are a special case - the shell converts the shorthand to an absolute path (`/home/training`) for you.

In the rest of this tutorial we will:

- assume the directory `/home/training/Linux` is our working (current) directory, unless otherwise stated
- write `~` instead of `/home/training`

In this section we have learned the following commands and symbols:

`cd` *`dir`*	change directory to subdirectory *`dir`*
`cd ..`	change directory to parent directory
`cd`	change directory to home directory
`~`	shorthand for home directory
`.`	shorthand for working (current) directory

Exercises on file system navigation

(1) Open a new shell, which will put you in your home directory.

(a) List the contents of the `Linux` directory. Do you need to use `cd` to do this?

(b) What about changing to the `Linux` directory and then listing the contents of your home directory without changing back?

(2) Using `--help`, learn how the `-a` option affects the `ls` command.

(3) Using the `man` command, find the option to sort the list of files by modification time when using `ls` command (Hint: search the `ls` man page for the word "modification").

(4) Look at the diagram showing the directory structure in section 3. Write out the **absolute** paths to the following directories: `home` (i.e. the directory with the name "home"), `dev`, `training`, and `Linux`.

(5) Similar to above, starting from your home directory (`/home/training`), write out the **relative** paths to the following directories: `home` (i.e. the directory with the name "home"), `dev`, `training`, and `Linux`.

Answers to exercises

(1)(a)

```
$ cd Linux
$ ls
```

or, without changing directory

```
$ ls Linux
```

(1)(b)

```
$ cd Linux
$ ls ..
```

or `ls ~` or `ls /home/training`

(2)

```
$ ls --help
...
-a, --all  do not ignore entries starting with .
...
```

Tip: try running `ls -a` in your home directory.

Files and directories starting with a `.` are not shown unless the `-a` flag is used. They are referred to as hidden files and directories (or dotfiles). Aside from the directory shortcuts `.` and `..` these mostly contain configuration settings and temporary files belonging to applications you run.

(3)

```
$ man ls
...
-t     sort by modification time, newest first
...
```

Remember - to quit the manpage, use the `[Q]` key.

Tip: try running `ls -alt` in your home directory.

(4)

```
home: /home
dev:  /dev
training: /home/training
Linux: /home/training/Linux
```

(5)

```
home: ..
dev:  ../../dev
training: .
Linux: Linux
```

5. Moving, copying and deleting files

Reminder

Make sure that you are in the directory `~/Linux` when starting a new chapter, unless it's explicitly stated that you have to be in another directory!

```
$ cd ~/Linux
```

Now we will have a look at how to create, copy, move, and remove files and directories within the shell.

First create two new directories, `temp1` and `temp2`, using the **`mkdir`** (make directory) command:

```
$ mkdir temp1 temp2
```

And check whether the directories have indeed been created:

```
$ ls
temp1  temp2
```

Change to the `temp1` directory:

```
$ cd temp1
```

Create two (empty) files, `myfile1.txt` and `myfile2.txt`, using the **`touch`** command:

```
$ touch myfile1.txt myfile2.txt
```

And check whether the files have been created:

```
$ ls
myfile1.txt myfile2.txt
```

Files can be copied using the **cp** (copy) command:

```
$ cp myfile1.txt ~/Linux/temp2
```

Because we copied it, the `temp1` dirbectory still contains the file `myfile1.txt`:

```
$ ls
myfile1.txt myfile2.txt
```

Files can be moved using the **mv** (move) command:

```
$ mv myfile2.txt ~/Linux/temp2
```

Because we moved it, the `temp1` directory doesn't contain the file `myfile2.txt` anymore:

```
$ ls
myfile1.txt
```

The `mv` command is also the command we use to rename files:

```
$ mv myfile1.txt myfile1_renamed.txt
$ ls
myfile1_renamed.txt
```

Specifying `cp` and `mv` targets

The `cp` and `mv` commands are entered in the format:

`command` *`source target`* (e.g. `cp myfile1.txt ../temp2`).

Note that:

If the target is an **existing directory**, the command will create a file with the same name as the source in the target directory. In this case you may have multiple source files.

If the target is an **existing file**, the command will **overwrite** the target file.

If the target **does not exist**, the command will create a new file with that name.

Files can be removed using the **`rm`** (remove) command:

```
$ rm myfile1_renamed.txt
$ ls
```

Go back to the directory `~/Linux`:

```
$ cd ~/Linux
```

Directories can be removed using the **`rmdir`** (remove directory) command. However, `rmdir` can only remove empty directories.

Thus, directory `temp1` can be removed:

```
$ rmdir temp1
$ ls
temp2
```

But directory `Temp2` cannot be removed this way, as it contains two files:

```
$ rmdir temp2
rmdir: failed to remove ‘temp2’: Directory not empty
$ ls
temp2
```

No news is good news

You'll see that the above `rmdir` command showed an error message, but the previous commands (`rm`, `mv`, `cd`, `rmdir`) just returned straight to the prompt. This is a tenet of the classic UNIX design - if a command worked as expected and has nothing unusual to report it says nothing. The core system commands all follow this pattern.

Often you can add a **`-v`** flag to get a verbose output, saying what was done.

One way to remove a non-empty directory is to first remove all the files it contains using `rm`, and then the directory itself using `rmdir`.

A quicker way is to remove the directory and its contents in one go using `rm` with the **`-r`** (i.e. recursive) option:

```
$ rm -r temp2
$ ls
```

So, `rmdir` may seem redundant but it is safer than `rm -r`, in that it will never remove any files, only empty folders.

In this section we have learned the following commands:

`mkdir` *`dir1 [dir2...]`*	create directory *`dir1 [dir2...]`*
`touch` *`file1 [file2...]`*	create empty file *`file1 [file2...]`*
`cp` *`file1 dir1`*	copy *`file1`* to *`dir1`*
`mv` *`file1 dir1`*	move *`file1`* into *`dir1`*
`mv` *`file1 file2`*	rename *`file1`* to *`file2`*
`rmdir` *`dir1 [dir2...]`*	remove empty directory *`dir1 [dir2...]`*
`rm` *`file1 [file2...]`*	remove *`file1 [file2...]`*
`rm -r` *`dir1 [dir2...]`*	recursively remove directory *`dir1 [dir2...]`* and its contents

Warning

When you remove or overwrite files from the command-line, they are gone. Forever. They are not put in a Trash or Recycle bin like in a graphical environment, so there is no easy way to get them back!

You can of course make your own ~/Trash directory and move things to there instead of deleting them, then empty out the trash when you are sure. This is good practise.

You can also manipulate files via the Graphical User Interface (GUI) browser (we won't tell!) but this may not always be available, and when you are already in the right directory in your shell it is quicker to type a command than navigate in the GUI.

Finally you can add the `-i` flag to `cp`, `mv` and `rm` so you will be specifically asked if you really want to remove each file. "`-i`" here is short for "interactive" On some systems this may be set as the default, but you should never rely on it.

Get into the habit of using the safe `rmdir` when you think the directory is empty, and triple-checking whenever you use `rm -r`. A "classic mistake" is to type something like:

```
$ rm -r ~/myproject /temp_files.
```

That extra space in the command means the entire `~/myproject` directory is going to silently vanish, followed by an error message saying that `/temp_files` cannot be found.

The danger is especially high if you are in a root shell (an administrative shell where file permissions, as described below, are not enforced). People have destroyed their whole systems with a single mistyped `rm -r` command. After this, recovery from backup is the only option.

6. File ownership and permissions

In this course you are working on your own personal machine image, but more typically a bioinformatician will work on a shared system, and you will want to share files with (and protect them from) other users.

Not everyone is allowed to read, edit, or execute every file or directory. What a person is allowed to do with a file or directory is determined by its **ownership** and **permissions**.

Each file (and directory) has three sets of permissions:

- **user**: Permissions for a user owning the file/directory.
- **group**: Permissions for a user group associated with the file/directory. Users may belong to several groups but each file has only one group.
- **others**: Permissions for everyone else, i.e. a user who is not the owner nor a member of the associated group.

Each file has three basic permission settings:

- **read** (`r`): Ability to read the contents of a file.
- **write** (`w`): Ability to write to or edit a file.
- **execute** (`x`): Ability to execute a file (i.e. run the file as a program).

For directories:

- **read** (`r`): Ability to list the contents of a directory.
- **write** (`w`): Ability to add/rename/delete the contents of a directory (useless without execute permission).
- **execute** (`x`): Ability to go into a directory and access files inside it

So for a directory with only **execute** permission you can read the contents but only if you already know the names of the files, because you cannot list them. This is occasionally useful but in general on any directory you want to set **read** and **execute** permissions both on, or both off.

To illustrate this, create a file `myfile.txt`:

```
$ touch myfile.txt
```

Listing the content of your home directory using `ls` with the long-listing option `-l` added shows details about all subdirectories and files, including their permissions:

```
$ ls -l
...
-rw-rw-r--  1 training training    0 Feb 27 11:47 myfile.txt
```

The first column (`-rw-rw-r--`) contains the permissions.

- The **`-`** at the start indicates that this is a file. If it was a **`d`** it would be a directory, if it was an **`l`** it would be a link (see the paragraph on symbolic links a little further down).
- The next three letters are the **user** permissions. `rw-` means that the user (you) can read the file, can write to/edit the file, but cannot execute it.
- The next three letters are the **group** permissions. `rw-` means that members of the group (here, `training`, as the default group name happens to be the same as the user name) can read the file, can write to/edit the file, but cannot execute it (even if it did contain executable code).
- The next three letters are the **others** permissions. `r--` means that everyone else can read the file, but cannot write to/edit or execute it.

Verbose output

The commands in the next section will be run with the **`-v`** (verbose) option. This option, which was mentioned briefly earlier, is supported by many commands and tells the command to 'describe' its performed actions (i.e. be verbose what it has done).

The `-v` option does not affect what a command actually does, only what messages it prints to the terminal.

To change the permissions of files, we use the **chmod** (change mode) command:

- The `chmod` command uses the letters `u`, `g` and `o` to represent user, group and others.
- The `chmod` command uses the letters `r`, `w` and `x` to represent read, write and execute.

For example, to stop other users on the system from accessing `myfile.txt`, you can remove the permissions for group and for others:

```
$ chmod -v g-rw myfile.txt
mode of ‘myfile.txt’ changed from 0664 (rw-rw-r--) to 0604 (rw----r--)
$ chmod -v o-r myfile.txt
mode of ‘myfile.txt’ changed from 0604 (rw----r--) to 0600 (rw-------)
```

To add permissions, use a `+` instead of a `-`.

Setting permissions recursively is possible with the `-R` flag. The following is a very handy command. It means “make my current directory and everything in it readable and writable by me”. Note the uppercase `X` properly sets the directory permissions, remembering that we want both **read** and **execute** set on directories. If we used a lowercase `x` it would do this but also make all files executable, which we probably don't want.

```
$ chmod -R u+rwX .
```

Changing the **owner** and **group** of a file is done using the **chown** and **chgrp** commands. However, only the system administrator (aka root user) can ever run **chown** so it is only useful if you happen to be the administrator on the machine. **chgrp** is more useful in that you can change the group designation of your own files to any group you are a member of (the **groups** command lists these).

```
$ groups
users audio video plugdev

$ chgrp users ~/Pictures
```

Now any group permissions set on your Pictures directory will apply to members of the `users` group.

If you need to take ownership of a file that is owned by another user, and you do not have the permission to run **chown**, you'll simply need to copy the file. The copy that you make will belong to you.

Symbolic links

Alternatively referred to as a **soft link** or **symlink**, a **symbolic link** is a pseudo-file that links to another file or directory using its path (rather like a shortcut in Microsoft Windows).

Symbolic links are created using the `ln` command with the `-s` option. The `-r` option is also useful as it makes `ln` behave more similarly to `cp` and `mv`:

```
$ ln -sr ~/Linux/mouse_exons.bed ~/Public
```

Symlinks are shown in long-format directory listings with an `l` in front and a `->` that points to the target file:

```
$ ls -l ~/Public
```

```
lrwxrwxrwx 1 training training 29 Feb 27 13:55 mouse_exons.bed ->
../Linux/mouse_exons.bed
```

Deleting the symbolic link does not remove the original file. If, however, the target file to which the link points is moved or removed, the link will stop working.

In this section we have learned the following commands:

`ls -l`	list contents of current directory in long list format
`chmod` *`perms file`*	modify permissions of *`file`*
`chgrp` *`newgroup file`*	set the group for *`file`*
`ln -sr` *`file1 dir1`*	make a symbolic link to *`file1`* in *`dir1`*

Exercises on file management and permissions

(1) Go to the directory `~/Linux`. In this directory, create a directory named `directory1`.

Within `directory1`, create two subdirectories, named `subdirectory1` and `subdirectory2`.

In `subirectory1`, create two empty files, named `file1` and `file2`.

Use the **tree** command to visualise what you have created. We've not used this command yet, but it's basically a version of **ls** that lists directories and their contents at once. You could use the relevant manpage to discover how it works, or just run it and see. You should see something a little like this:

```
/home/training/Linux/
...
 directory1/
  subdirectory1/
   file1
   file2
  subdirectory2/
```

(2) View the permissions of `file1` and `file2`. Change the permissions of `file1`, so that the user (you) can read and write, the group can read, and others cannot do anything.

(3) Copy `file1` and `file2` from `subdirectory1` to `subdirectory2`. (**hint** - if you want to copy both files in one go, you can use the `*` wildcard). Check whether both files have been copied.

(4) Find the appropriate options for the **`tree`** command to show the file permissions, users and groups and use this to view the whole directory structure, as in part 1. Did the copied files in `subdirectory2` retain the permissions you set in part 2?

(5) Stay in the directory `~/Linux`. Remove `subdirectory1` and the files it contains with a single command. Check whether the directory has been deleted.

Answers to exercises

(1) This is one way to do it...

```
$ cd ~/Linux
$ mkdir directory1
$ cd directory1
$ mkdir subdirectory1
$ mkdir subdirectory2
$ cd subdirectory1
$ touch file1
$ touch file2
```

```
$ cd ~/Linux
$ tree directory1
...
```

It's actually possible to make all the directories and files in just two commands, but this uses a shell feature we've not encountered yet, so nobody would expect you to come up with the following. We just want to show it's possible:

```
$ mkdir -p directory1/subdirectory{1,2}
$ touch directory1/subdirectory1/file{1,2}
```

(2)

```
$ cd ~/Linux/directory1/subdirectory1

$ ls -l
-rw-rw-r-- 1 training training 0 Feb 27 14:05 file1
-rw-rw-r-- 1 training training 0 Feb 27 14:05 file2
```

```
$ chmod g-w
$ chmod o-r file1
$ ls -l
-rw-r----- 1 training training 0 Feb 27 14:05 file1
-rw-rw-r-- 1 training training 0 Feb 27 14:05 file2
```

(3) Again, this is just one possible way...

```
$ cd ~/Linux/directory1/subdirectory1
$ cp file1 ../subdirectory2
$ cp file2 ../subdirectory2
```

or with the * wildcard

```
$ cp file* ../subdirectory2
```

```
$ ls ../subdirectory2
file1 file2
```

(4)

```
$ man tree
...
-p   Print the file type and permissions for each file (as per ls -l).
-u   Print the username, ...
-g     Print the group name, ...
...
```

To quit the man page, use the `[Q]` key.

```
$ cd ~/Linux
$ tree -pug directory1
directory1
├── [drwxrwxr-x training training]  subdirectory1
│   ├── [-rw-r----- training training]  file1
│   └── [-rw-rw-r-- training training]  file2
└── [drwxrwxr-x training training]  subdirectory2
    ├── [-rw-rw-r-- training training]  file1
    └── [-rw-rw-r-- training training]  file2

2 directories, 4 files
```

(5)

```
$ rm -r directory1/subdirectory1
```

or (two commands, but safer!):

```
$ rm directory1/subdirectory1/*
$ rmdir directory1/subdirectory1
```

```
$ tree -pug directory1
directory1
└── [drwxrwxr-x training training]  subdirectory2
    ├── [-rw-rw-r-- training training]  file1
    └── [-rw-rw-r-- training training]  file2

1 directory, 2 files
```

7. Viewing text files

So far we've moved empty files around and completely ignored the file contents. There are several core commands to access the contents of a file.

The file we are having a look at in this section is in **BED** format and contains the genomic locations of mouse exons. The first few lines of the file look like this:

```
chr19   3153798   3154009     NR_045304_exon_0_0_chr19_3153799_r  0     -
chr19   3196944   3197029     NR_045304_exon_1_0_chr19_3196945_r  0     -
chr19   3197604   3197703     NR_045304_exon_2_0_chr19_3197605_r  0     -
chr19   3259075   3261635     NM_009212_exon_0_0_chr19_3259076_r  0     -
```

Each line represents one exon, showing the chromosome, start position, end position, etc. More about the BED format later.

Since the BED format is a type of text file and not a binary format you can display and filter it using standard Linux tools, even though none of these tools were written specifically with BED files in mind

The **`head`** command prints the first few lines of the file to the terminal:

```
$ head mouse_exons.bed
```

This is very useful to peek at a large file. If you want to see the whole file you can use the **`cat`** (concatenate) command, but if the file is large it will just whizz up the screen too fast to read.

```
$ cat mouse_exons.bed
```

The **`less`** command (the name is another pun - an improved version of the older **`more`** command) allows you to look at the file content one screen at a time:

```
$ less mouse_exons.bed
```

You can scroll through the BED file one line at a time using the `[Down arrow]` and `[Up arrow]` keys, or one screen at a time using the `[Page Down]` key or space bar, and the `[Page Up]` or `[B]` key. To access the help, use the `[H]` key, and `[Q]` to quit.

If the keyboard shortcuts in `less` seem familiar, this is because the `man` command actually uses `less` to show the manual page content. We've been using `less` all along.

The commands **`less`** and **`more`** are called "pagers" because they show files a page at a time.

Much like **head**, the **tail** command just returns the last few lines of the file:

```
$ head mouse_exons.bed
$ tail mouse_exons.bed
```

By default, `head` and `tail` return ten lines. This can be changed using the **-n** option:

```
$ head -n 5 mouse_exons.bed
```

The **wc** (word count) command returns the number of lines, words, and bytes in a file:

```
$ wc mouse_exons.bed
4592 27546 289638 mouse_exons.bed
```

If you're only interested in the number of lines, add the `-l` (lines) option:

```
$ wc -l mouse_exons.bed
4592 mouse_exons.bed
```

In this section we have learned the following commands:

`cat` *file*	output content of *file*
`less` *file*	output content of *file* one screen at a time
`head` *file*	output first 10 lines of *file*
`head -n` *num file*	output first *num* lines of *file*
`tail` *file*	output last 10 lines of *file*
`tail -n` *num file*	output last *num* lines of *file*
`wc` *file*	print number of lines, words, and bytes in *file*
`wc -l` *file*	print number of lines in *file*

8. Downloading remote files

On the training VMs you can run the Chrome web browser, but not all Linux systems will be set up to support this (eg. on Eddie access is command-line only). Files that you would normally access via a web browser also can be downloaded using the command **`wget`** (World Wide Web get).

➔ Download a file from the ENA Sequence Read Archive (http://www.ebi.ac.uk/ena):

```
$ wget ftp://ftp.sra.ebi.ac.uk/vol1/fastq/SRR026/SRR026762/SRR026762.fastq.gz
...
```

The progress of the download will be displayed. Depending on the size of the file and the speed of your network connection, this can take a little while, but you will see a progress bar in the terminal.

The file contains sequence reads from a miRNA study in human embryonic stem cells in **gzipped FASTQ** format (more about this format this afternoon).

In this section we have learned the following command:

`wget` *`file`*	download *`file`*

9. Zipping and unzipping files

The file we downloaded in the last chapter is a compressed **gzip** (GNU zip) file. Compressed files use less disk space to store and can be transferred faster, and are commonly encountered in bioinformatics.

Have a look at the size of the file:

```
$ ls -lh
...
-rw-rw-r--    1    training    training    162M    Feb    27    14:22
SRR026762.fastq.gz
...
```

Note that with the option `-h` (human readable format) file/directory sizes are shown using unit suffixes: Byte, Kilobyte, Megabyte, Gigabyte, etc. So this file is taking up 162 megabytes on disk.

Unzip (de-compress) the file using the `gunzip` command:

```
$ gunzip SRR026762.fastq.gz
```

Note that the zipped file has now gone, and have a look at the size of the uncompressed file:

```
$ ls -lh
...
-rw-rw-r-- 1 training training 656M Feb 27 14:22 SRR026762.fastq
...
```

Use the `head` command to peek at the first few lines of the file:

```
$ head -n 6 SRR026762.fastq
@SRR026762.1 HS00992..1:5:1:660:893/1
GAATTCTACAGTCCGACGATCGTATGCCGTCTTATG
+
IIIIIIIIIHIIIII(I0I@I:%III.II4IGI%D-
@SRR026762.2 HS00992..1:5:1:544:535/1
GGGGATGTAGCTCAGTGGTAGAGTCGTATGCCGTCT
```

The FASTQ format is a standard way of storing raw sequencer reads. Each read is represented by

four lines - a header, the sequence itself, a '+' spacer line and then a string encoding the quality of each basecall. So the box above shows the first record and half of the second one.

Now zip up the file again using the **gzip** command:

```
$ gzip SRR026762.fastq
```

And see what happens when we run **head** on the compressed file:

```
$  head -n 6 SRR026762.fastq.gz
*◆◆◆◆◆*1u%y◆Y◆◆◆6◆◆◆K◆>◆:◆x◆◆K◆|Ϗ◆5j1z-◆8◆◆◆`◆◆Db◆J◆◆◆M◆◆◆◆◆◆i◆◆>i
◆6i◆2n◆&;$◆◆n◆◆◆◆a◆◆◆◆◆◆◆0v◆◆◆◆◆◆?◆◆◆.◆◆J◆◆]◆◆◆◆^◆\
%¾(>◆?◆◆◆◆◆◆w1◆◆o◆◆F◆N◆h◆wTon◆B◆◆◆y.◆◆◆◆m◆◆$◆◆t◆◆◆B◆ก◆)u◆_}◆◆3◆◆◆R
H◆◆◆◆◆9=☒◆◆◆a◆◆wW◆◆z◆◆◆◆.◆◆/◆◆:Rj◆◆◆◆JH◆◆◆6◆9RS◆%◆◆◆0◆X◆s◆◆6ob◆◆◆L
<A-◆◆:◆◆,◆_n◆1~n◆kZ◆}◆◆◆◆R◆X◆◆◆Ut◆',◆wU◆l◆◆\b◆◆◆hee◆]ŵ◆◆◆:<◆[◆ő◆◆◆0
K[uau@dR◆}◆g    "C◆◆◆u5◆◆◆Hav◆&k◆&E◆◆◆@QQKg◆:◆|)ol◆
```

Ewwww! Printing a binary file to the terminal is neither useful nor pretty.

To just have a look at the contents of a gzipped file, it isn't necessary to unzip it, as modern versions of the **less** command actually knows how to deal with compressed files:

```
$ less SRR026762.fastq.gz
```

In fact, in most cases it is never necessary to **gunzip** a compressed file before reading the contents, because we can combine **zcat** with other commands in one go, as we'll see in the next chapter.

Like `cat`, `zcat` prints the whole content of the file to the screen, but it decompresses it on-the-fly. You can try running `zcat` on the sample file but it will take a while to print out as the file is large. To interrupt the process, use `[Ctrl]+[C]`.

A **tar** (tape archive) file, with extension `.tar`, is another common file format whereby entire directory structures and all of the files within them have been placed into a single file. Normally these are then compressed with **gzip** so you get a file with the extension `.tar.gz` (sometimes `.tgz`). Tar files, compressed or not, can be extracted using the **`tar -xaf`** command, eg:

```
$ wget https://ftp.gnu.org/gnu/tar/tar-latest.tar.gz
$ tar -xaf tar-latest.tar.gz
```

where **`-x`** means extract files from an archive, **`-a`** automatically handles the decompression and **`-f`** specifies the archive file to read. It's also normal to add **`-v`**, the flag verbose for verbose output, so you get to see a list of what is being unpacked.

Other archive file formats

Linux tar files perform exactly the same function as zip files more normally used in Windows, and Linux can also pack and unpack .zip files as well as any other compressed format you are likely to encounter. Likewise, software to handle tar files is widely available for Mac and Windows so you should never have a problem exchanging files whatever format you use.

A reason for sticking to tar format on Linux is that tar knows about file ownership, permissions, symlinks and other quirks of the Linux filesystem.

In this section we have learned the following commands:

`gunzip file.gz`	decompress `file.gz` to `file`
`gzip file.gz`	compress `file` to `file.gz`
`zcat file.gz`	output content of `file.gz`
`tar -xvaf file.tar.gz`	extract `file.tar.gz` listing the files as they are extracted

10. Pipes and redirects

Some of the real power of the Linux shell comes from the ability to string commands together to form "shell pipelines".

With the `|` (pipe) operator (which you can find on most keyboards at the bottom left next to the `[Shift]` key), the output of one command, written on the left of the pipe, can be used as input for another command, placed on the right of the pipe.

For example, if you want to count up the number of lines in the gzipped file `SRR026267.fastq.gz` (see previous chapter) you can do this by first using `zcat` to get the real uncompressed content of the file, and then **piping** the output of `zcat` into `wc` to count the lines and characters in the file:

```
$ zcat SRR026762.fastq.gz | wc
```

If you simply use `wc` on the file, you'll get a result but it will be nonsense as `wc` does not know how to decompress the file before counting the contents. You might think that the `wc` command should be enhanced to work on gzipped files, but with pipes there is no need; you can simply join the commands together to make your own.

With the `>` (redirect) operator, the output of a command can be saved in a file, rather than being shown in the terminal.

For example, let's say we want to create a file `my_commands.txt`, that contains the text "My commands:", followed by the contents of our command history.

To this end, first write a line saying "`My commands:`" to a new file. This can be done using the **`echo`** command, which simply prints out any text that you give it, then redirect this to the file:

```
$ echo My commands: > my_commands.txt
```

To check the content of the `my_commands.txt` file, `cat` it to the screen:

```
$ cat my_commands.txt
My commands:
```

If we now add the contents of our command history to the file using the `>` operator, it turns out that the original content of the file is overwritten and the first line is gone:

```
$ history > my_commands.txt
$ head my_commands.txt
    1 echo Hello World!
    2 ls
    3 history
    ...
```

To add output to the contents of an already existing file without overwriting it, we should use the `>>` (append) operator instead:

```
$ echo My commands: > my_commands.txt
$ history >> my_commands.txt
$ head my_commands.txt
My commands:
    1 echo Hello World!
    2 ls
    3 history
    ...
```

In this section we have learned the following shell features:

`echo any text you like`	print back the given text in the terminal
`command1 | command2`	use output of *`command1`* as input for *`command2`*
`command > file`	redirect result of *`command`* to *`file`*
`command >> file`	append result of *`command`* to end of *`file`*

Exercises on viewing data in files

(1) Download the file `Mus_musculus.GRCm38.dna.chromosome.Y.fa.gz` from the Ensembl FTP site. This file contains the reference sequence of the mouse Y chromosome in **FASTA** format, which means there is a single header line and then many lines of sequence.

The URL for the file is:

`ftp://ftp.ensembl.org/pub/release-84/fasta/mus_musculus/dna/Mus_musculus.GRCm38.dna.chromosome.Y.fa.gz`

(If you're viewing the digital version of this doc you should be able to copy the URL and paste it into the shell to download the file with wget.)

(2) Have a look at the uncompressed file contents. Try using the '`-n`' option of the `cat` command to add line numbers to the file before you view it.

Note that the sequence starts with 11060 lines of N's, representing the unsequencable end of the chromosome.

(3) Write the portion of the sequence from line 11055 to 11064 inclusive to a new file, named `Mus_musculus_Y_10.txt`. The new file should contain 10 lines.

(4) How much disk space is being saved by storing this file in gzip compressed format?

(5) Going back to the `SRR026762.fastq.gz` file you downloaded in chapter 8, how many sequence records are in the file?

(6) If, in completing any of the above exercises, you used the `gunzip` command to unpack the compressed file on disk, see if you can answer the question without doing this.

Hint 1: You can have multiple pipes and redirects in one line.

Hint 2: The `gunzip` command can tell you the real size of a file without unpacking it

Answers to exercises

(1) Use wget or the regular web browser for this.

(2) Viewing the file is pretty simple.

```
$ less Mus_musculus.GRCm38.dna.chromosome.Y.fa.gz
```

We could also do the following, uncompressing the file explicitly then piping the text into `less`. On the systems we are using this is redundant, but on older Linux systems the `less` command may not be able to understand the compressed file, but using `zcat` and the pipe explicitly will always work.

```
$ zcat Mus_musculus.GRCm38.dna.chromosome.Y.fa.gz | less
```

This also leads us to a solution that numbers the lines and views them in one go.

```
$ zcat Mus_musculus.GRCm38.dna.chromosome.Y.fa.gz | cat -n | less
```

Or in this case there's an even easier way - the manpage for `less` tells us that "`less -N`" adds line numbers, so we could just do that.

```
$ less -N Mus_musculus.GRCm38.dna.chromosome.Y.fa.gz
```

(3)

This one really does need pipes. The trick here is to use the `head` and `tail` commands to chop out the first 11064 lines of the file and then take the last 10 of those. You could save the intermediate results to temporary files or just chain pipes to do it in one go. (Note - this is one single command line but is just wrapped to fit on the page).

```
$ zcat Mus_musculus.GRCm38.dna.chromosome.Y.fa.gz |
    head -n 11064 | tail -n 10 > Mus_musculus_Y_10.txt
```

(4) You could uncompress the file and use `ls` to see the size, but a neater option is to just ask `gzip`.

```
$ gzip -l Mus_musculus.GRCm38.dna.chromosome.Y.fa.gz
 compressed  uncompressed  ratio  uncompressed_name
 26655784    93273832      71.4%  Mus_musculus.GRCm38.dna.chromosome.Y.fa
```

The space saving is `93273832 - 26655784 = 66618048` bytes, ie. ~63 megabytes. These files compress really well because there is so much repetition within the sequence.

(5)

```
$ zcat SRR026762.fastq.gz | wc -l
22710524
```

Divide by 4 to get the number of sequences, as there are four lines in the file per sequence, so 22710524 / 4 is 5677631, or ~5.7 million.

11. Filtering lines and columns from files

Have a look at the file `Mus_musculus_snps_19_1_20000000.vcf`:

```
$ less Mus_musculus_snps_19_1_20000000.vcf
```

This file contains information about variants on the first 20 Mb of mouse chromosome 19 and is in **VCF** format.

VCF (Variant Call Format) (https://samtools.github.io/hts-specs/VCFv4.2.pdf) is a standardised text file format to store variants. It contains meta-information lines, then a single header line, and then data lines each containing information about a position in the genome. (VCF files may also contain variant calls for individual samples, but this particular file does not.)

VCF files are often stored in a compressed manner as `.vcf.gz`. You have already seen how to read compressed files using zcat and pipes, but for simplicity here we just have the file uncompressed.

The 12 meta-information lines all start with "`##`".

The header line starts with a single "`#`" and names the 8 fixed columns. These are as follows:

`CHROM`: chromosome
`POS`: position (from start of chromosome)
`ID`: variant identifier
`REF`: reference base(s)
`ALT`: alternate base(s)
`QUAL`: quality (not used in our file)
`FILTER`: filter status (not used in our file)
`INFO`: additional information – in our file this field contains the source (always dbSNP) and variant type (SNV, insertion, or deletion)

Fields in both the header line and data lines are separated by tab characters.

Let's say we want to extract from this file a **list of all known insertion sequences**. (Note that for insertions and deletions, both the `REF` and the `ALT` field start with the base that is present before the insertion or deletion event.)

This task is a bit more complex than what we have done until now, but can be built up from simple steps. The basic process consists of four steps, for each of which we will introduce a different Linux utility:

- Extract all lines that represent an insertion using **`egrep`**.
- From these lines extract the sequence of the insertion using **`cut`**.
- Order the sequences using **`sort`**.
- Remove any redundant/duplicate sequences using **`uniq`**.

egrep is a utility to filter text files for lines matching a search pattern. egrep prints out a file to the terminal, much like cat, but in addition to the file name requires a search pattern argument, and will print only the lines containing that pattern as output.

To find all lines in Mus_musculus_snps_19_1_20000000.vcf that contain the word "insertion" we can run egrep like so:

```
$ egrep insertion Mus_musculus_snps_19_1_20000000.vcf | head
19 3100169 rs250477339 T TC . . dbSNP_138;TSA=insertion
19 3101934 rs243804858 G GC . . dbSNP_138;TSA=insertion
19 3118475 rs387027980 G GATGTATGTATGTATGTATGT . .
dbSNP_138;TSA=insertion
19 3118519 rs211854236 C CAC . . dbSNP_138;TSA=insertion
19 3124385 rs261495216 A AGATAGATAGATA . . dbSNP_138;TSA=insertion
...
```

Note how we previewed the output while avoiding spewing the whole thing to the screen by piping into head. We could have piped to less instead.

We can count how many insertions we have by piping the results into wc instead:

```
$ egrep insertion Mus_musculus_snps_19_1_20000000.vcf | wc -l
30621
```

-

grep, egrep and regular expressions

If you've previously heard of the **grep** command you may wonder why we are talking about **egrep** instead of "regular" **grep**. Using **egrep** simply activates an extended syntax for regular expressions which is slightly simpler to work with, and more compatible with other tools, so in general there's no reason not to use **egrep**.

The real power of egrep comes from these **regular expressions**, which you can make as sophisticated as you want:

words like "insertion" just match that exact text
. matches any single character
? the preceding character matches 0 or 1 times
* the preceding character matches 0 or more times
+ the preceding character matches 1 or more times
{n} the preceding character matches exactly n times
{n,m} the preceding character matches at least n times and not more than m times

`{n,}` the preceding character matches at least n times
`{,m}` the preceding character matches not more than m times
`[agd]` the character is one of those included within the square brackets
`[^agd]` the character is not one of those included within the square brackets
`[c-f]` the dash within the square brackets operates as a range, in this case meaning any of the letters `c`, `d`, `e`, or `f`
`(la)` group several characters together, so `(la){3}` matches `lalala`.
`|` the logical OR operation
`^` matches (anchors) the beginning of the line
`$` matches the end of the line

In general, to prevent the shell from trying to interpret the search patterns before they get to **egrep**, they need to be put in 'single quotes'.

Example:

The restriction enzyme `StyI` cuts at the sequence `5'-CCWWGG-3'`, where `W = A or T` (`W` stands for "weak", because `A-T` has only two H-bonds).

To find occurrences of this restriction site in a DNA sequence the following `egrep` command can be used:

```
$ egrep 'CC[AT]{2}GG' file_containing_dna_sequence
```

Note that there are often many ways to accomplish the same thing. The above pattern can for example also be written as `'CC(A|T)(A|T)GG'` or `'C{2}(A|T){2}G{2}'` etc.

cut is a utility to cut out selected columns (e.g. fields) of each line of a file. It assumes by default that fields are tab-delimited, which is handy for VCF files.

In our file the sequence of each insertion is stored in the fifth field (ALT). By using a pipe we can extract the fifth field from the results we obtained using `egrep`. Again using `head` gives us just a preview of the result:

```
$ egrep insertion Mus_musculus_snps_19_1_20000000.vcf | cut -f 5 | head
TC
GC
GATGTATGTATGATGTATGT
CAC
AGATAGATAGATA
...
```

The **uniq** command is, once again, a little like cat except that if it sees a repeated line it will only print the line once. Since uniq only recognises and removes adjacent lines that are identical, our list of sequences first has to be sorted using **sort**:

```
$ egrep insertion Mus_musculus_snps_19_1_20000000.vcf | cut -f 5 | sort | head
AA
AA
AA
AA
AA
...
```

By default sort sorts alphabetically, but it is also possible to sort numerically. More information about this and other options can be found on the sort manpage. Here the default will be fine.

Now after sorting any redundancy can be removed using uniq:

```
$ egrep insertion Mus_musculus_snps_19_1_20000000.vcf | cut -f 5 | sort | uniq |
head
AA
AAA
AAAA
AAAAA
AA, AAAA
...
```

Count again how many sequences we have:

```
$ egrep insertion Mus_musculus_snps_19_1_20000000.vcf | cut -f 5 | sort -d |
uniq | wc -l
5826
```

As a sanity check, we can do the same with the fourth field (REF), which should only contain the four bases A, C, G, and T:

```
$ egrep insertion Mus_musculus_snps_19_1_20000000.vcf | cut -f 4 | sort | uniq
A
C
G
T
```

We are nearly done, but there are three considerations left:

1. Saving the list of unique insertion sequences to a file
2. Correctly handling lines that have two alternative sequences, like "AA , AAAA"
3. Removing the first base which, as noted above, is not actually part of the insertion

These final steps are presented as an exercise below.

In this section we have learned the following commands:

`egrep` *`pattern file`*	search for *`pattern`* in *`file`*
`cut` *`file`*	removes part of lines of *`file`*
`sort` *`file`*	sort lines in *`file`*
`uniq` *`file`*	remove duplicate lines from sorted *`file`*

Exercises on filtering text files

As we noted previously, the file `Mus_musculus_snps_19_1_20000000.vcf` contains variants on the first 20 kb of mouse chromosome 19. The file is in VCF format.

(1) Have a look at the file. Find an example of each of the three types of variants, i.e. SNVs, insertions, and deletions.

(2) Ensure that you can run the command given at the end of the previous chapter, ie:

```
$ egrep insertion Mus_musculus_snps_19_1_20000000.vcf | cut -f 5 | sort -d
    | uniq | wc -l
```

You should get the answer 5826.

(a) Instead of counting the lines, how would you save the list of results to a file named mm_insertions.txt?
(b) How many lines in this file contain more than one alternative sequence (ie., they contain a comma)?
(c) Find a way to properly handle these lines. (*Hint: check the manpage of egrep and specifically the '-o' option, and you'll need to use a regular expression pattern)*
(d) Remove the first base from each sequence in the list, since this actually represents the base before the insertion. (*Hint: the cut command can do this for you*)

(3) Have a look at the section about regular expression patterns in `egrep`. Also ensure you have the **mm_insertions.txt** file from exercise 2. This file should have 4424 lines in it - if not, run the sample answer for question 2 to get the right input file. Answer each of the following with a command of the form:

```
egrep 'PATTERN' mm_insertions.txt | wc -l
```

(a) How many of the sequences in the file start with ATG?
(b) How many of these are followed by between 5 to 10 more bases after the ATG?
(c) How many sequences contain only A's and T's?
(d) How many contain a codon for leucine followed by a glutamine, in any of the possible forward reading frames? (Leucine=CTT,CTC,CTA,CTG,TTA,TTG; Glutamine=CAA,CAG).

Answers to exercises

(1) You could simply look at the file in `less`, adding line numbers...

```
$ less -N Mus_musculus_snps_19_1_20000000.vcf
```

Then press the / key to search for "SNV", "insertion" and "deletion". The first SNV is on line 14, the first insertion on line 88 and the first deletion is on line 100.

To quit `less`, use the `[Q]` key.

(2)

(a) Use the > operator after all the pipes. The following is a single command line:

```
$ egrep insertion Mus_musculus_snps_19_1_20000000.vcf | cut -f 5 |
sort -d | uniq > mm_insertions.txt
```

(b) Use egrep to look for the commas and wc to count the lines. The answer is 42.

```
$ egrep ',' mm_insertions.txt | wc -l
```

(c) One idea is simply to filter out these lines. This can be done with:

```
egrep -v ','
```

That is, using egrep to find lines that contain a comma, but then using `-v` to filter them out.

But to actually include all the sequences in the list we can say, for example:

```
egrep -o '[ATCG]+'
```

The pattern [ATCG]+ matches a string of one or more nucleotides, and the `-o` flag tells egrep to only output the match, and not the entire line. Crucially, with `-o`, if egrep sees multiple matches on one line it will print them all, so it effectively splits the lines for us.

We need to do the filtering step before the sorting step in the pipeline.

```
$ egrep insertion Mus_musculus_snps_19_1_20000000.vcf | cut -f 5
   | egrep -o '[ATCG]+' | sort | uniq > mm_insertions.txt
```

(d) The `-c` option to the cut command allows you to extract characters rather than fields, and then "`cut -c 2-`" gives us everything from character 2 onwards. To be meaningful, this must be done after splitting the lines with commas but before sorting.

```
$ egrep insertion Mus_musculus_snps_19_1_20000000.vcf | cut -f 5
    | egrep -o '[ATCG]+' | cut -c 2- | sort | uniq >
mm_insertions.txt
```

This will give you the final result with 4424 lines in the file.

(3)

(a) Search pattern `^ATG finds 56 lines`

(b) `^ATG.{5,10}$ finds 23 lines`

The ^ and $ are needed to ensure the pattern matches the entire line - equivalently you could use `egrep -x` which adds both anchors implicitly.

(c) `^[AT]+$ finds 245 lines`

(d) `(CTT|CTC|CTA|CTG|TTA|TTG)(CAA|CAG) finds 44 lines.`

The pattern can be written more compactly as:

```
(CT[ATCG]|TT[AG])CA[AG]
```

12. Aligning sequences to a reference genome

We have now used some standard text processing tools within the shell and demonstrated how they can be applied to some file types (FASTQ, FASTA and VCF) that a bioinformatician would encounter. There are many more useful commands in the standard Linux shell toolkit, and in the last chapter of this course we'll take a brief look at **sed** and **awk**, a pair of general purpose text processing commands which give you the power to manipulate text files in any number of ways.

Obviously, not all bioinformatics tasks on Linux can be accomplished without specialist tools so now we'll have a look at a fairly typical task in bioinformatics. Given some raw DNA sequencing reads, how do we align these to an existing reference genome, and what output should we expect from doing this? To really simplify things, we'll make the following choices:

1. We'll skip any pre-processing of the reads, even though they probably contain sequencing adapter and low quality basecalls that should ideally be trimmed out.
2. We'll use the popular **bowtie2** aligner software. Many other aligners are available, but all of them are run in much the same way and produce **SAM** formatted output.
3. We'll use the **samtools** package to view and summarize the SAM file. Other toolkits are available - Picard Tools being the most notable.
4. We'll not attempt to call variants or do any further analysis. This is a whole course in itself!

Within the `Linux/sequence_alignment` directory the files `6991_1.fastq.gz` and `6991_2.fastq.gz` contain the raw reads. Just like the FASTQ file we examined in chapters 8 and 9 they contain four lines of text for each read. Looking at the files in `less` you can see the reads are 45 bases long. Each of the files contain 1.9 million reads and the corresponding reads are paired between the two files, ie. each read in file `6991_1` is from the 5' of one fragment and each corresponding read in file `6991_2` is from the 3' end of the same fragment. This is standard for short read sequencing, and is just what the bowtie2 aligner expects.

These reads are all from E. coli, and the reference genome we'll try to align against is in the file `Escherichia_coli_bw25113.ASM75055v1.dna.toplevel.fa.gz`. The file is in gzipped FASTA format.

Before we can align the reads, we need to index the genome. Some aligners don't require this step but bowtie2 does. The point is that indexing large genomes can be slow and we only want to have to do it once. Our little E. coli genome will be indexed in just a few seconds.

```
$ bowtie2-build Escherichia_coli_bw25113.ASM75055v1.dna.toplevel.fa.gz
      Ecoli_index
```

If you list the directory you will see six new .bt2 files have appeared. These will be used by the bowtie2 aligner in the next step. We're now ready to align the reads, but to speed things up we'll just align the first 10% of the reads. We can extract these using another standard shell tool, **split**.

```
$ zcat 6991_1.fastq.gz | split -d -l 769264 - 6991_ --add=_1.fastq
$ zcat 6991_2.fastq.gz | split -d -l 769264 - 6991_ --add=_2.fastq
```

This should produce 20 individual FASTQ files named `6991_00_1.fastq`, `6991_01_1.fastq`, etc. The magic number **769264** is just the number of lines in the files (as obtained by wc) divided by 10.

Now to align the `6991_00_1.fastq` and `6991_00_2.fastq` files:

```
$ bowtie2 -x Ecoli_index -1 6991_00_1.fastq -2 6991_00_2.fastq -S 6991_00.sam
```

This should only take a few seconds. Note that we needed to tell bowtie2 the index name, as given to bowtie2-build, as well as the input files to be aligned and the output file to save. There are many other options for running the alignment but we're keeping things super-simple for this demonstration.

The output file is another type of tabular text file and we can take a look at it:

```
$ less -S 6991_00.sam
```

(The `-S` option just tells `less` not to wrap long lines - lines in SAM files are normally very long.)

Just by eye, we can see that the reads are in pairs (look at the sequence names in the first column) and that some are mapped and others are not (looking at columns 3 to 9). We could attack the file with `grep` and `cut` just as with the VCF file previously, but instead we'll use a package that is specific to handling SAM file data - **samtools**.

```
$ samtools flagstat 6991_00.sam
384632 + 0 in total (QC-passed reads + QC-failed reads)
0 + 0 secondary
0 + 0 supplementary
0 + 0 duplicates
276681 + 0 mapped (71.93% : N/A)
384632 + 0 paired in sequencing
192316 + 0 read1
192316 + 0 read2
267174 + 0 properly paired (69.46% : N/A)
270768 + 0 with itself and mate mapped
5913 + 0 singletons (1.54% : N/A)
0 + 0 with mate mapped to a different chr
0 + 0 with mate mapped to a different chr (mapQ>=5)
```

Looking at this output, there are lots of zeros because we have not asked bowtie2 to make secondary alignments or mark duplicate reads or apply QC criteria, and also because there is only one chromosome in the genome. With other options or other aligners we may see more complex results. The main point of information here is that 71.93% of our reads mapped and 69.46% of our read pairs properly mapped (ie. one to each strand).

Now we have a SAM file, let's try and determine the coverage depth. Doing this by eye from looking at the SAM file in `less` is a non-starter, so what we need is a stacked view of the reads along the genome. Let's try and view the alignment with '`samtools tview`':

```
$ samtools tview 6991_00.sam
samtools tview: cannot read index for "6991_00.sam"
```

This command says it needs an index. So let's try and generate one:

```
$ samtools index 6991_00.sam
samtools index: "6991_00.sam" is in a format that cannot be usefully indexed
```

Hmmm. This is a rather obtuse error message from samtools, but it actually means we need to convert the SAM file to BAM format. BAM files are binary SAM files - they contain exactly the same data but in a compressed binary format which is more efficient for the computer to store and read. We also need to re-order the reads by the position they map to the genome, rather than the order they came off the sequencer. These are standard steps after mapping, and can be done with a single command to samtools:

```
$ samtools sort -o 6991_00.bam 6991_00.sam
```

And we still need to make the index on the BAM file, which should now work:

```
$ samtools index 6991_00.bam
```

A small binary file named 6991_00.bam.bai will have been created. Now to view:

```
$ samtools tview 6991_00.bam
```

Hold Shift+Right to pan quickly along the genome. The individual fragments are represented in the positions where they mapped. Along the top, under the location numbers, you'll see "NNNNNN . . .". This line would normally show the reference sequence but we didn't explicitly tell samtools about our original FASTA reference which is why it is showing N's. We can do that now -

press Q to quit the viewer and:

```
$ samtools tview 6991_00.bam
    Escherichia_coli_bw25113.ASM75055v1.dna.toplevel.fa.gz
```

Now the reference genome is shown for us in place of the N's along the top, and then in the fragments below only bases which differ from the reference are shown, with the rest being replaced with dots and commas. Panning along the genome, can you start to see loci where our sequenced strain varies from the reference?

We can see by eye that we have about 3x coverage on average. Let's get samtools to report the actual coverage.

```
$ samtools stats 6991_00.bam | less
```

This produces a long report, which is why we piped the output into less, allowing us to scroll through it. Go right to the bottom (it's a long way - use `[Shift+G]` to jump right there) and you'll see a histogram of coverage. The modal coverage is 2x, at 879515 positions. You'll notice that samtools neglects to report the number of positions with zero coverage, or to report an overall mean. This is a quirk of samtools. A general rule in bioinformatics is that it's very rare for a single tool to give you all the information that you need, which is why bioinformaticians use a whole array of software, and sometimes have to write their own.

Clearly we've just scratched the surface here, using one aligner with the default settings, and we've not even started the actual analysis of the data to characterize variants, compare between samples, examine gene annotations, etc. etc. This is all beyond the scope of this course.

In this section we met the following commands:

`bowtie2-build` *`genome index_name`*	index the given *`genome`* for use with the bowtie2 mapper
`split`	a standard Linux command that splits a file into chunks
`bowtie2`	maps short reads to a pre-indexed genome and produces a SAM file as output
`samtools`	a toolkit that provides various operations applicable to SAM (or BAM) files

13. Installing new bioinformatics tools

In the previous chapter we generated a SAM file using the bowtie2 aligner. Virtually all read

mapping tools (bowtie, bwa, STAR, minimap2, etc.) will output in SAM format. We also used the BAM format which is a binary encoding of the SAM format, using the **samtools** package to perform the conversion and obtain some statistics about the alignments.

In this chapter we'll look at **bedtools**. This 'swiss army knife' of bioinformatics tools can be used for a wide variety of tasks. It has great documentation including a helpful website: https://bedtools.readthedocs.io/en/latest/index.html .

Unlike samtools, bedtools is not currently installed on our systems so let us use this opportunity to install a new package to our machines. To do this we will use the Ubuntu package manager. All Linux distributions have their own package manager, and for Debian-derived systems, like Ubuntu, we can interact with the package manager using the **`apt`** command (APT = Advanced Packaging Tool). This performs such functions as installation of new software packages, upgrading of existing ones, updating of the package list index, and even upgrading the entire Ubuntu system. Let's try '`apt install`' to download and install bedtools.

```
$ apt install bedtools
E: Could not open lock file /var/lib/dpkg/lock-frontend - open (13: Permission denied)
E: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontend), are you root?
```

An error message appears: You do not have permission to do this: you're not the "root" user, and only the "root" user has permission to change files in the system directories.

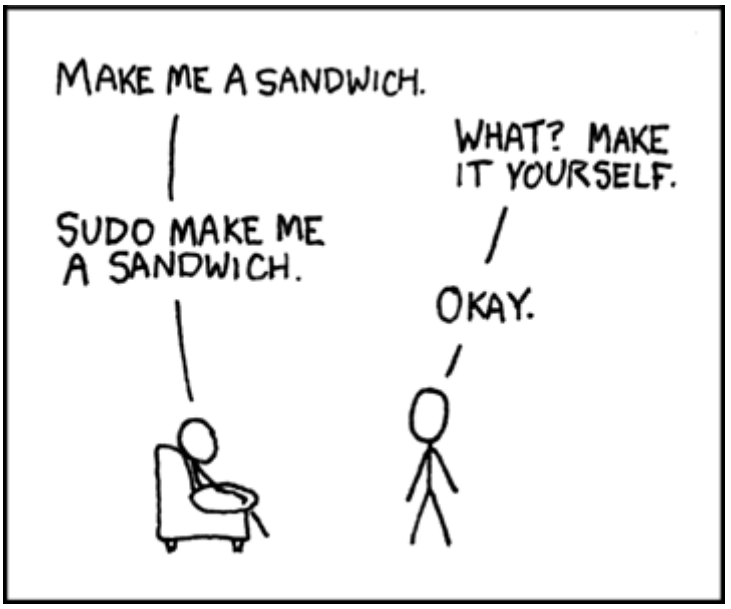
Don't worry, on your training VM you can access the power of the root account using **`sudo`**. Standing for **S**uper **U**ser **Do**, `sudo` lets you access the privileges of the system root user or 'Super User'. Elevating your privileges is often necessary to carry out jobs like installing new software. So, let's add `sudo` to the beginning of the command and try again to install bedtools using our newfound power.

```
$ sudo apt install bedtools
Reading package lists... Done
Building dependency tree
Reading state information... Done
...
Setting up bedtools (2.26.0+dfsg-5) ...
```

Ta-Da! Sudo gives us elevated privileges when we want to run administrative commands: However, be aware sudo is not a cure-all for permissions problems and thoughtless usage can potentially get you into trouble. Most likely if your computer requires extra permission to do something there's a good reason for that; altering files important to the running of your operating system for example. Even if you are the owner of the computer and have full sudo access, consider exactly why you are

using sudo before you use it!

With great power comes great responsibility: There is no su-undo!



We can also view and manage what packages are installed or available to install via the package manager with a GUI called `synaptic`. This command has to be run with `sudo` if you want to make changes to installed packages or install anything new.

```
$ sudo synaptic
```

You can use the search function in the synaptic GUI to check that bedtools is installed but if we want a further sanity check we can always just type `bedtools` on the command line, if it's installed we will get a ream of information:

```
$ bedtools

bedtools is a powerful toolset for genome arithmetic.

Version:   v2.26.0
About:  developed in the quinlanlab.org and by many contributors worldwide.
Docs:   http://bedtools.readthedocs.io/
Code:   https://github.com/arq5x/bedtools2
Mail:   https://groups.google.com/forum/#!forum/bedtools-discuss
```

The exercises below explore some uses of bedtools.

Both Bedtools and Samtools can convert BAM files back to FASTQ, losing the header information and any alignments in the process. Another useful general format conversion tool is **seqtk,** which

can be found at https://github.com/lh3/seqtk. Like bedtools, seqtk is available to install as a package on the training VMs. Seqtk also supports trimming, subsampling, etc. The examples on the website give a good idea of the capabilities of this tool, and see exercise 3 below for a practical example of using it.

In this section we met the following commands:

`sudo` *`command`*	run a command with administrator-level (aka. root) privileges
`sudo apt install` *`package_name`*	fetch and install the named package to the local system, assuming it exists in the package repository
`sudo synaptic`	provides a GUI interface to the package manager
`bedtools`	another example of a bioinformatics toolkit, which mostly deals with BED formatted files

Exercises on bedtools, samtools, etc

The exercises in this section are somewhat more open-ended than in previous sections. You will need to consult relevant documentation outside of this course book and possibly search around for hints on the internet. There may be several right answers. A little trial and error also goes a long way.

1) How many exon sequences do not overlap with any other exon sequence in the BED file `~/Linux/mouse_exons.bed`?

Tip: Use your newly installed bedtools package to find the 'intersect' of this file with itself.

2) Type the command **apt source samtools.** A directory named `samtools-1.7` should be created in your current working directory.

Try to use **samtools flagstat** on the sample file in `samtools-1.7/examples/ex1.sam.gz`.

a) What error do you get and what does this signify? Can you work out how to make it work and obtain the stats?

b) What similar statistics can you obtain from a SAM file using bedtools, and what first step is necessary in order to do this?

3) Use **seqtk** to convert the `SRR026762.fastq.gz` file you downloaded earlier in the course into FASTA format, having trimmed low quality bases. (You will need to start by looking for instructions on https://github.com/lh3/seqtk then installing **seqtk**, either with APT or by downloading it from GitHub.)

4)

a) Work out how to use the **split** tool (as seen in chapter 13) to split the mouse_exons.bed file into 10 chunks. Make sure you don't split the file in the middle of a line. Also, ensure the separate files you create have names that start with "mouse_exons.bed".

b) Search for information on the **GNU Parallel** tool. Get it running on your VM, then use it to count the lines (with **wc**) in all the chunks, but counting four chunks at once in parallel. (Note that this is a very fast operation on this particular file, but for larger jobs this would allow you to make use of multiple CPU cores in the machine at once in order to speed up a large job.)

Answers to Exercises

1) For this problem we have to think a bit laterally: comparing our file to itself using bedtools 'intersect'. We use the -c option to count the number of overlapping features.

```
$ bedtools intersect -a ~/Linux/mouse_exons.bed
    -b ~/Linux/mouse_exons.bed -c
```

Features in this output that only overlapped with themselves will be counted as one in the final column of the row. We can use grep to select only those lines that have a one at the end '1$' and then count the number of lines in that output like so:

```
$ bedtools intersect -a ~/Linux/mouse_exons.bed
    -b ~/Linux/mouse_exons.bed -c | grep "1$" | wc
```

This should give you a line count of **2335** which represents the number of exons in our bed file that do not overlap with anything other than themselves.

2) The error is "missing SAM header". You can verify this by looking at the file in **less.** There are no lines at the top starting with **@.** (The fact that the file is compressed with gzip is not relevant here).

This signifies that the SAM file is incomplete, and lacks header information on the target genome, so no statistics can be produced by Samtools.

To fix the problem we need to add a header. There is a file called `00README.txt` in the `examples` directory that spells this out so the real test here is did you spot that? If a file says `README` it's often worth reading!

This file suggests a solution, using the original "genome" in `ex1.fa` to build an index which can be used to construct the SAM header. Then flagstat will work.

```
$ cp samtools-1.7/examples/ex1.* .
$ samtools faidx ex1.fa
$ samtools view -h -t ex1.fa.fai ex1.sam.gz | samtools flagstat -
3307 + 0 in total (QC-passed reads + QC-failed reads)
0 + 0 secondary
```

In the newer versions of **samtools** this can actually be done in one shot, though the

`00README.txt` doesn't mention it:

```
$ samtools view -h -T ex1.fa ex1.sam.gz | samtools flagstat -
[samfaipath] build FASTA index...
3307 + 0 in total (QC-passed reads + QC-failed reads)
0 + 0 secondary
...

$ ls
ex1.fa  ex1.fa.fai  ex1.sam.gz
```

3) Using APT:

```
$ sudo apt install seqtk
```

Otherwise, after downloading and unzipping the source code and running **make** you should have a working **seqtk** program. The exact command you need to run will depend on the location of this file:

```
$ ~/seqtk-master/seqtk trimfq SRR026762.fastq.gz |
    ~/seqtk-master/seqtk seq -a - > SRR026762.trimmed.fasta
```

4a)

```
$ split -n l/10 mouse_exons.bed mouse_exons.bed.
```

4b) Parallel homepage is at: https://www.gnu.org/software/parallel/

You can "sudo apt install parallel" or else the appropriate download is:

https://www.mirrorservice.org/sites/ftp.gnu.org/gnu/parallel/parallel-latest.tar.bz2

You can follow the instructions for local installation or simply place the executable in your home dir. Once you have the "parallel" command, one solution is:

```
$ ls mouse_exons.bed.?? | parallel -j 4 wc
```

14. Shell scripts and loops

Instead of entering and executing commands one-by-one on the command-line, it is also possible to store a sequence of commands in a text file and tell Bash to execute these in one go. This is known as a **shell script**.

To write a shell script we need a **text editor**. There exist many text editors. Some popular ones are for example `Emacs`, `gedit`, `nano/pico`, and `vi`. We will use the `nano` text editor, included in most Linux distributions. It runs within the terminal.

Open `nano` from the command-line:

```
$ nano
```

Our shell script should start with a special line indicating that it is to run in Bash:

```
#!/bin/bash
```

Followed by the command(s) to be executed. We'll start with something really simple:

```
echo This is my first shell script!
```

To save, press `[Ctrl]+[O]`, name the file `my_first_script.sh`, and press `[Enter]`. It will be saved to your working directory, presumably `~/Linux`.

You can exit `nano` by pressing `[Ctrl]+[X]`.

Now do `ls -l`:

```
$ ls -l
...
-rw-rw-r-- 1 training training 36 Feb 27 15:15 my_first_script.sh
...
```

This shows that the shell script has been created, but according to the permissions (`rw-rw-r--`) it isn't executable yet.

Make the script executable for the user:

```
$ chmod u+x my_first_script.sh
```

And check again:

```
$ ls -l
...
-rwxrw-r-- 1 training training 15 Jan 9 14:37 my_first_script.sh
...
```

Now the script can be executed:

```
$ ./my_first_script.sh
This is my first shell script!
```

Us of the "./" syntax says "look in the working directory for my script rather than looking at all the directories listed in $PATH".

The $PATH variable

Whenever we issue a command, we actually run a program (similarly to how we just ran our script). Because these programs can be in several locations in the file system and it is not feasible for us to remember the path to each one of them, we use $PATH to simply call these programs by their names.

$PATH is an **environment variable** that tells the shell where to look for executable files in response to commands issued by a user.

We can examine the $PATH variable using the echo command:

```
$ echo $PATH
```

```
/home/training/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

As you can see, $PATH is simply a list of directories that will be searched for our command. For example, when we type pwd, the shell will first look for an executable file called pwd in /home/training/bin, then /usr/local/sbin, etc. etc. Once it has found a match, it will execute that file.

In order to find the actual location of a command, you can use **which**:

```
$ which pwd
/usr/bin/pwd
```

If the command you type includes a **/** anywhere in the name then the shell will ignore the $PATH and just follow the normal rules to find the file by relative/absolute path. This is why `./my_first_script.sh` runs our script, even though it is not in the $PATH.

As well as listing multiple commands, one very useful thing that you can put in your script is a **shell loop**. Shell loops may be used outside of scripts too, but in scripts they are particularly useful. Consider the following rather trivial script.

```
#!/bin/bash
echo a
echo b
echo c
```

If you saved the above as `script1.sh`, you could run it and print out three letters.

```
$ ./script1.sh
a
b
c
```

Because we are doing exactly the same thing for each letter, we can convert this to a loop.

```
#!/bin/bash
for a_letter in a b c ; do
  echo $a_letter
done
```

There's a lot to take in here, but this is the standard template for all shell loops:

1. The loop starts with the word **for**
2. The next word, "a_letter" is a variable name which you choose. It could be "x" or "foo" or whatever you like.
3. Then the keyword **in**
4. Then a list of items to loop through, terminated by a semi-colon
5. The the keyword **do**
6. Then the commands to be run in the loop - indenting the lines just makes it easier to read
7. Finally the keyword **done**

Inside the loop, if you put `$a_letter` (or `$x` or `$foo` or whatever name you chose) the shell

will substitute the value of the loop item. So in this case the loop runs three times and prints each letter, just like the first trivial script. Let's do something a little more useful. The file `drosophila_species_cleaned.txt` contains a list of Drosophila species copy-pasted from Wikipedia. It's a plain text file so we can extract lines from it with `egrep`. Here is a loop that runs `egrep` three times.

```
#!/bin/bash
for a_letter in a b c ; do
  egrep "^D\. $a_letter" drosophila_species_cleaned.txt > dros_$a_letter.txt
done
```

As when using `egrep` previously, we need to put quotes around the search pattern. As long as we use double quotes, the shell will still substitute variables like `$a_letter`. If single quotes had been used the substitution would not happen and `egrep` would literally try to search for the string 'a_letter' within the file. The pattern above therefore extracts all the species with names that begin with 'a', then 'b', then 'c', and is equivalent to:

```
#!/bin/bash
egrep "^D\. a" drosophila_species_cleaned.txt > dros_a.txt
egrep "^D\. b" drosophila_species_cleaned.txt > dros_b.txt
egrep "^D\. c" drosophila_species_cleaned.txt > dros_c.txt
```

Note that as well as using the loop variable to set the search pattern, it's being used to make a filename where the output will be redirected.

If we want to extract more letters, we only need to add them at the top of the loop, rather than copying the whole line.

```
#!/bin/bash
for a_letter in a b c d e f g h i j k l m ; do
  egrep "^D\. $a_letter" drosophila_species_cleaned.txt > dros_$a_letter.txt
done
```

The most common use of shell loops is to loop over a list of input files. To avoid listing all the filenames one by one, we can use glob patterns. So to count the number of species in all of the output files made by the above script we could write this loop:

```
#!/bin/bash
for a_file in dros_*.txt ; do
  echo "Counting species in $a_file..."
  wc -l $a_file
done
```

The structure is the same as before, but here we let the shell expand the pattern `dros_*.txt` to provide the list of values for `a_file`, and the shell runs two commands for each of the files it finds - `echo` and `wc`.

Writing more complex scripts is a big subject and beyond the scope of this course. For some more on scripts have a look a this online resource:

https://linuxconfig.org/bash-scripting-tutorial-for-beginners

In this section we have learned the following commands:

`nano`	open the `nano` text editor
`./script.sh`	run `script.sh` (assuming it is present in the working directory and executable)
`echo` *`$VAR`*	Show the value of a loop or environment variable, for example: `echo $PATH`
`echo "VAR is` *`$VAR`*`"`	Insert the value of a variable into a double quoted string and print the result
`which` *`command`*	show if a command is available in the $PATH and if so, where the file is
`for` *`x`* `in` *`list`* `; do` *`cmds`* `; done`	loop through a *`list`* of words and run *`cmds`* for each word in turn

Exercises on shell scripting

(1) Once again this exercise uses the file `Mus_musculus_snps_19_1_20000000.vcf` first encountered in chapter 11. The file is in VCF format.

(a) Take a look at the file to remind yourself what it contains

(b) Open `nano`

(c) Write a shell script that uses `egrep` and `wc` to count the SNVs, insertions, and deletions in the file, and gives the following output:

```
Number of SNVs:
371291
Number of insertions:
30621
Number of deletions:
35835
```

(d) Save the script as `count_variants.sh`

(e) Make the script executable.

(f) Run the script.

(2) This exercise uses the example files from chapter 12, found under `~/Linux/sequence_alignment`. If you have not done so already, follow the example commands in that chapter to split the original files into ten chunks and align the first of these (`6991_00_1.fastq` and `6991_00_2.fastq` files) to make a SAM file.

a) Make a script that will align all ten of the chunks in a loop and report some alignment statistics for each chunk.
b) Work out how to combine all of the ten outputs into a single sorted BAM file using samtools, and add this as a final step of your script.

Answers to exercises

(1)(a)

```
$ less Mus_musculus_snps_19_1_20000000.vcf
```

To quit `less`, use the `[Q]` key.

(b)

```
$ nano
```

(c) In the `nano` editor:

```
#!/bin/bash
echo Number of SNVs:
egrep SNV ~/Linux/Mus_musculus_snps_19_1_20000000.vcf | wc -l
echo Number of insertions:
egrep insertion ~/Linux/Mus_musculus_snps_19_1_20000000.vcf | wc -l
echo Number of deletions:
egrep deletion ~/Linux/Mus_musculus_snps_19_1_20000000.vcf | wc -l
```

(d)

`[Ctrl]+[O]`
File Name to Write: `count_variants.sh`
`[Enter]`
`[Ctrl]+[X]`

(e)

```
$ chmod u+x count_variants.sh
```

(f) Remember to use `./` to run a script or program in the current working directory.

```
$ ./count_variants.sh
Number of SNVs:
371291
Number of insertions:
30621
Number of deletions:
35835
```

2) This loop would work. The choice of what statistics to calculate the result is up to you but I chose to get the count of properly mapped reads. Lines ending "\" are continued on the next line:

```
for f_prefix in 6991_00 6991_01 6991_02 6991_03 6991_04 \
                6991_05 6991_06 6991_07 6991_08 6991_09 ; do
    bowtie2 -p 2 -x Ecoli_index -1 "$f_prefix"_1.fastq \
                                -2 "$f_prefix"_2.fastq -S "$f_prefix".sam
    samtools flagstat "$f_prefix".sam | egrep proper
done
```

This is a fuller solution which starts with the splitting operation then uses a glob pattern match for the alignment and merge steps. It also uses a couple of features not mentioned above:

- Error trapping is turned on with "`set -euo pipefail`"
- Getting the prefix (first 7 characters) of the filenames with "`f_prefix=${f:0:7}`"

```
#!/bin/bash
set -euo pipefail

for x in 1 2 ; do
  zcat 6991_"$x".fastq.gz | split -d -l 769264 - 6991_ --add=_"$x".fastq
done

for f in 6991_??_1.fastq ; do
    f_prefix=${f:0:7}
    bowtie2 -p 2 -x Ecoli_index -1 "$f_prefix"_1.fastq \
                                -2 "$f_prefix"_2.fastq -S "$f_prefix".sam
    samtools flagstat "$f_prefix".sam | egrep -proper
done

samtools merge -n 6991.bam 6991_??.sam
samtools flagstat 6991.bam
```

15. Process management

Sometimes you will want to stop a process you started. There are several ways to do this.

Download the primary assembly of the human genome in FASTA format from the Ensembl ftp site:

```
$ wget
ftp://ftp.ensembl.org/pub/release-87/fasta/homo_sapiens/dna/Homo_sapiens.GRCh38.
dna.primary_assembly.fa.gz
```

As this is a large file (840 MB), downloading will take quite a while.

Unzip the file:

```
$ gunzip Homo_sapiens.GRCh38.dna.primary_assembly.fa.gz
```

Note that there is no command prompt while the unzipping is going on. Consequently, unless we open a new terminal window, we cannot run any other commands until the unzipping has been completed.

To terminate the unzipping process, press the `[Ctrl]+[C]` keys.

To be able to keep working in the current terminal window while the unzipping is going on, add the `&` (background) operator to your command. This means "run the command in the background", i.e. detach it from the terminal window it was started from, so it does not block the command-line:

```
$ gunzip Homo_sapiens.GRCh38.dna.primary_assembly.fa.gz &
[1] 18329
```

When starting a process in the background, its process ID (PID) is printed (in this case 18329).

All active processes and their process IDs can be displayed using the `top` command:

```
$ top
```

To quit the top page, use the `[Q]` key.

Processes running in the background cannot be stopped using `[Ctrl]+[C]`. To stop them, use the **`kill`** command and their process ID:

```
$ kill 18329
```

If you forget to put the **`&`** operator on the end of your command, it is still possible to "push" the job into the background and get your shell prompt back. This is a two step process, as you first have to **suspend** the job with `[Ctrl]+[Z]` and then set it running again in the background with `bg`.

```
$ zcat *.fa.gz | wc
^Z
[1]+  Stopped        zcat primary_assembly.fa.gz | wc
$ bg
[1]+ zcat primary_assembly.fastq.gz | wc &
$
```

In this section we have learned the following commands:

`[Ctrl]+[C]`	stop a process that runs in the foreground
`command` &	run *`command`* in the background and show the *`PID`*
`top`	show all active processes
`kill` *`PID`*	stop process with *`PID`*
`[Ctrl]+[Z]`	suspend a command that was running in the terminal and show the prompt
`bg`	set the suspended command running again, but in the background

16. AWK and SED

Anyone who uses the Linux shell should at least be aware of **awk** and **sed**. Both commands allow you to filter the contents of files, like **egrep** and **cut**, but the operations you can apply are virtually unlimited. For example you can:

- Find and replace text in FASTA headers
- Perform arithmetic on numeric columns in a SAM file
- Extract info from headers in a SAM or VCF file
- Print only the sequence lines from a FASTQ file
- Split a single FASTA file into files of individual contigs

This power derives from the fact that instead of providing just a search pattern or a list of columns, both awk and sed take a script which will be applied to each line of the input file. The syntax of these scripts is peculiar to the tools themselves and is designed to be super-compact to fit on the command line.

awk (named after its creators, **A**ho, **W**einberger and **K**ernighan) is most useful for files that contain data tables, which includes most of the biological data files we have encountered so far.

The basic syntax is:

```
$ awk '/search_pattern/ {action_to_take_on matches}' file
```

Note that the part in single quotes is one single argument that is passed to awk. It is enclosed in single quotes to tell Bash not to try and interpret it at all. If the awk script gets long and complex it can instead be put in a file of its own, just like a shell script.

For example, to retrieve from the file `Mus_musculus_snps_19_1_20000000.vcf` all substitutions (SNVs):

```
$ awk '/SNV/ {print}' Mus_musculus_snps_19_1_20000000.vcf | head
19 3014737 rs261379297 T C . . dbSNP_138;TSA=SNV
19 3015490 rs234736085 T G . . dbSNP_138;TSA=SNV
19 3016625 rs253681447 G A . . dbSNP_138;TSA=SNV
19 3016634 rs215663680 A G . . dbSNP_138;TSA=SNV
19 3122634 rs226523430 C T . . dbSNP_138;TSA=SNV
...
```

This could be done with `egrep`, of course, but apart from searching for a pattern in an entire line, it is also possible to test only a particular field for a pattern.

The first, second, third etc. column in the file are referred to by variables `$1`, `$2`, `$3` etc. For example, to retrieve all `A` to `T` substitutions (SNVs):

```
$ awk '/SNV/ && $4 == "A" && $5 == "T" {print}'
     Mus_musculus_snps_19_1_20000000.vcf | head
19 3039321 rs230672390 A T . . dbSNP_138;TSA=SNV
19 3090604 rs246554770 A T . . dbSNP_138;TSA=SNV
19 3090903 rs214999397 A T . . dbSNP_138;TSA=SNV;E_MO
19 3094489 rs250447880 A T . . dbSNP_138;TSA=SNV
19 3101306 rs227960563 A T . . dbSNP_138;TSA=SNV
...
```

Basically, this awk statement means: for each line in the file `Mus_musculus_snps_19_1_20000000.vcf` that contains the text "SNV", and in which the fourth and fifth field match "A" and "T" respectively, print the entire line.

Note that `$4 == "A"` tests that the entire field matches `"A"`. To just test for the occurrence of an `"A"` within the field, use the pattern matching operator `$4 ~ /A/` instead.

To print out only the identifiers for the `A` to `T` substitutions (which are stored in the third field) we can modify the print statement:

```
$ awk '/SNV/ && $4 == "A" && $5 == "T" {print $3}'
     Mus_musculus_snps_19_1_20000000.vcf | head
rs230672390
rs246554770
rs214999397
...
```

sed (Stream EDitor) uses an even more compact script syntax than awk, and does not split lines into fields. The most common use of `sed` is to perform a global find-and-replace on a text file.

```
$ sed 's/Hardy/Medd/g' drosophila_species_cleaned.txt
...
```

The **`'s///'`** syntax tells sed to replace anything that matches the first word (which may be a regex pattern) with the second word. The **`g`** tells `sed` that it should replace all occurrences on a line (g for global). You can make multiple substitutions in one pass - for this you need to use the **`-e`** option:

```
$ sed -e 's/Hardy/Medd/g' -e 's/Kaneshiro/Booth/g'
    drosophila_species_cleaned.txt > amended_credit.txt
```

Sed also provides a neat way to extract every, say, fourth line of a file, so to get just the sequence lines from a FASTQ file:

```
$ zcat SRR026762.fastq.gz | sed -n '2~4p' > sequences.txt
```

Many websites exist to share useful `awk` and `sed` one-liners, which you may be able to use right away or adapt to the task at hand, and whole books have been written on both tools. This course can only give a brief taster. Here are some good starting points for further reading:

https://catonmat.net/awk-one-liners-explained-part-one

https://catonmat.net/sed-one-liners-explained-part-one

Useful Resources

Directory of Linux commands (http://archive.oreilly.com/linux/cmd/):

A directory of 687 Linux commands (taken from Linux in a Nutshell, 5th Edition), with for each command a description and list of available options.

LinuxQuestions (http://www.linuxquestions.org):

A community-driven, self-help web site for Linux users. The most popular section of the site are the forums, where Linux users can share their knowledge and experience. Newcomers to the Linux world (often called newbies) can ask questions and Linux experts can offer advice.

Stack Overflow (http://stackoverflow.com):

A question-and-answer website on the topic of computer programming.

Biostars (http://www.biostars.org):

A question-and-answer website with a focus on bioinformatics, computational genomics and biological data analysis.

Before posting a question on one of the above websites, you should always first try to find the answer yourself by (1) doing a Google search and (2) searching the website for previously asked questions!

Also, even in this internet age, it's worth getting a good book.